

Nautilus Logistics AI — Deep Codebase Explanation

Exhaustive File-by-File Technical Guide & "Explain Like a Baby" (ELI5) Breakdown

SYSTEM VERSION
v2.0.0 Production

TECH STACK
Polyglot C++/Py/TS

DOCUMENT PURPOSE
Code & Data Flow Defense

TARGET AUDIENCE
Hackathon Jury & Developers

1. Layer Distribution & Architecture Overview

Nautilus Logistics AI is divided into 5 distinct operational layers. Below is the full file count and responsibility mapping.

LAYER	LANGUAGES	FILES	TECHNICAL FUNCTION	👤 SIMPLE ANALOGY (ELI5)
Frontend & UI	TypeScript, TSX, CSS3, HTML5	21 Files	Liquid Glass UI, Recharts graphs, Leaflet sea map, bilingual translations.	The Captain's Steering Wheel: The shiny dashboard with touch buttons, live maps, and easy-to-read dials.
AI & Analytics	Python 3.11	7 Files	30/60/90-day price forecasting, KAVACH weather swell risk, IMO carbon emissions.	The Weather & Price Fortune Teller: Smart robots looking at 10 years of data to tell you future prices and storm alerts.
C/C++ Core	C (C11), C++ (C++17)	8 Files	Microsecond Great Circle nautical math, azimuth bearings, voyage economics.	The Lightning Math Rocket: Calculating complex Earth geometry in microseconds so the app never freezes.
Launcher & Setup	C++17, Win32 API	10 Files	Zero-dependency desktop app launcher, instant in-memory package installer.	The Magic Tool Box: A single double-clickable file that installs and opens the app without needing internet or passwords.
Project Ops	JSON, TOML, Batch	13 Files	Vite bundler config, TypeScript rules, Netlify cloud deployment, build scripts.	The Factory Blueprint: Instructions and recipes that build, test, and package the whole project cleanly.

2. Frontend Architecture & UI Layer (src/ & public/)

FILE PATH	TECH	TECHNICAL PURPOSE (WHAT IT DOES)	👤 EXPLAIN LIKE A BABY (SIMPLE ANALOGY)	STATUS
<code>src/App.tsx</code>	TSX / React 19	Main application orchestrator. Controls parameter bar state (Commodity, Tonnage, Origin, Destination, Ship), KPI cards (Action Signal, Projected Savings, Confidence), 5D vessel optimizer, and Recharts forward curve rendering.	The Master Brain & Dashboard: Imagine playing a ship game. When you pick "Coking Coal" and "Paradip Port", this file instantly updates all the numbers, graphs, and recommendations on your screen in a split second!	CRITICAL
<code>src/main.tsx</code>	TSX / React 19	Application bootstrap entrypoint. Mounts the root React component tree into the HTML DOM using createRoot under StrictMode.	The Ignition Key: Like turning the key to start a car's engine. Without this file, the React code would never turn on or show up in the web browser.	HIGH
<code>src/SeaPortsMap.tsx</code>	TSX / Leaflet	Interactive geospatial maritime route map. Renders 18 Indian ports, 12 global gateways, great-circle trade lanes, and live vessel telemetry markers.	The Google Maps for Ships: Draws a big blue ocean map with pins on ports and lines showing where ships are sailing, plus toy boats moving across the water!	HIGH
<code>src/translations.ts</code>	TypeScript	Bilingual i18n dictionary supporting English and Hindi (हिन्दी). Localizes all metrics, labels, port names, vessel classes, and tooltips.	The Translator Robot: A giant phrasebook. When you click the Hindi button, it instantly swaps every English word into Hindi so anyone in India can use it easily.	HIGH

FILE PATH	TECH	TECHNICAL PURPOSE (WHAT IT DOES)	🧐 EXPLAIN LIKE A BABY (SIMPLE ANALOGY)	STATUS
<code>src/index.css</code>	CSS3 / Tailwind	Global styling stylesheet implementing Liquid Glassmorphic backdrop blur (blur(42px)), neon accents, ambient glows, and animations.	The Fashion Stylist & Makeup Artist: Gives the app its gorgeous glowing frosted-glass look, neon lights, dark blue ocean themes, and smooth animations.	HIGH
<code>src/components/sagarsetu/AISVesselTracker.tsx</code>	TSX / React 19	SagarSetu AIS fleet tracking component. Displays vessel names, MMSI, SOG (Speed Over Ground), headings, draft, and destination ETAs.	The Marine Radar: Like an air traffic control radar for airplanes, but for giant cargo ships out at sea, telling you how fast they are moving and when they will arrive.	HIGH
<code>src/components/sagarsetu/GreenFreightCarbon.tsx</code>	TSX / React 19	IMO MARPOL Annex VI carbon accounting card. Displays Scope-3 CO ₂ footprint, CII ratings (Class A–E), carbon offset surcharges, and Green Scores.	The Smoke Meter & Tree Counter: Measures how much dirty exhaust smoke the ship engine makes, gives the ship a school grade (A is great, E is bad!), and calculates how many green carbon credits to buy.	HIGH
<code>src/components/sagarsetu/KAVACHWeatherCard.tsx</code>	TSX / React 19	KAVACH meteorological risk intelligence card. Displays 3-point weather risk (Origin, Transit Corridor, Destination), wave heights, wind speeds, and delay premiums.	The Ocean Umbrella & Storm Guard: Checks the weather along the whole voyage. Warns if 4-meter giant waves or cyclones are coming so the captain can avoid danger!	HIGH
<code>src/components/OfflineNoticeModal.tsx</code>	TSX / React 19	Offline PWA modal alerting users when internet connectivity drops and confirming cached model execution.	The "No WiFi? No Worries!" Pop-up: If your internet dies while sailing at sea, this pop-up says: "Relax! Everything is already saved on your laptop and works 100% offline!"	MEDIUM
<code>public/sw.js</code>	JavaScript	PWA Service Worker script managing cache storage, network-first caching strategies, and offline asset availability.	The Magic Backpack: Secretly keeps a duplicate copy of all images and code in your computer's pocket so the app opens instantly with zero loading delay.	HIGH
<code>public/manifest.json</code>	JSON	Progressive Web App manifest declaring application name, standalone display mode, theme colors, and icons.	The App's Identity Card: Tells your phone/laptop: "I am a real installed app! Put my anchor icon on your Desktop!"	MEDIUM

3. AI & Machine Learning Engine (python_engine/)

FILE PATH	TECH	TECHNICAL PURPOSE (WHAT IT DOES)	🧐 EXPLAIN LIKE A BABY (SIMPLE ANALOGY)	STATUS
<code>python_engine/freight_predictor.py</code>	Python 3.11	Core time-series freight forecasting model. Generates 30, 60, and 90-day forward price curves, seasonal Fourier cycles, commodity volatility curves, and Bayesian confidence intervals ($\pm 3.2\%$ to $\pm 6.8\%$).	The Shipping Price Fortune Teller: Looks at 10 years of market history and says: "Hire a ship today at \$14/ton, because next month seasonal demand will push it up to \$17/ton!"	CRITICAL
<code>python_engine/weather_kavach.py</code>	Python 3.11	KAVACH meteorological AI. Computes 3-point composite oceanic risk score (25% Origin + 50% Sea Lane + 25% Destination), wave swell height, wind drag, and adverse sea-state delay surcharges.	The 3-Point Storm Calculator: Like checking the weather at home, on the highway, and at school. Combines all 3 into one single risk number (0 to 100%) so you know if your voyage is safe.	CRITICAL
<code>python_engine/carbon_cii.py</code>	Python 3.11	IMO MARPOL Annex VI carbon accounting module. Audits voyage fuel burn, computes Scope-3 emissions (3.114 t CO ₂ / t VLSFO), evaluates CII vessel grades (A–E), and calculates carbon offset costs.	The Pollution Tax Accountant: Knows that burning 1 ton of oil creates 3.114 tons of greenhouse gas. Tells the boss exactly how much pollution tax to pay.	HIGH
<code>python_engine/ais_tracker.py</code>	Python 3.11	AIS fleet telemetry engine. Ingests simulated and live NMEA AIS vessel positions, parsing speed over ground, heading, draught, and destination ETA across maritime trade lanes.	The Ship GPS Tracker: Listens to radio beeps from ocean ships and extracts where they are, how fast they are moving, and where they are going.	HIGH
<code>python_engine/c_cpp_bridge.py</code>	Python / ctypes	Foreign Function Interface (FFI) bridge linking Python with native compiled maritime_math.dll for microsecond geodesic distance and bearing calculations.	The Language Bridge: Python is smart and friendly, but C is super-fast like a Ferrari. This file is the bridge connecting them together with zero speed loss.	HIGH
<code>python_engine/server.py</code>	Python 3.11	Local development HTTP web server with automated port conflict discovery, explicit MIME-type mapping, and SPA fallback routing.	The Local Web Waiter: Serves the website to your browser from your own computer, making sure all images and scripts load without errors.	HIGH

4. High-Performance C/C++ Engine (c_cpp_engine/)

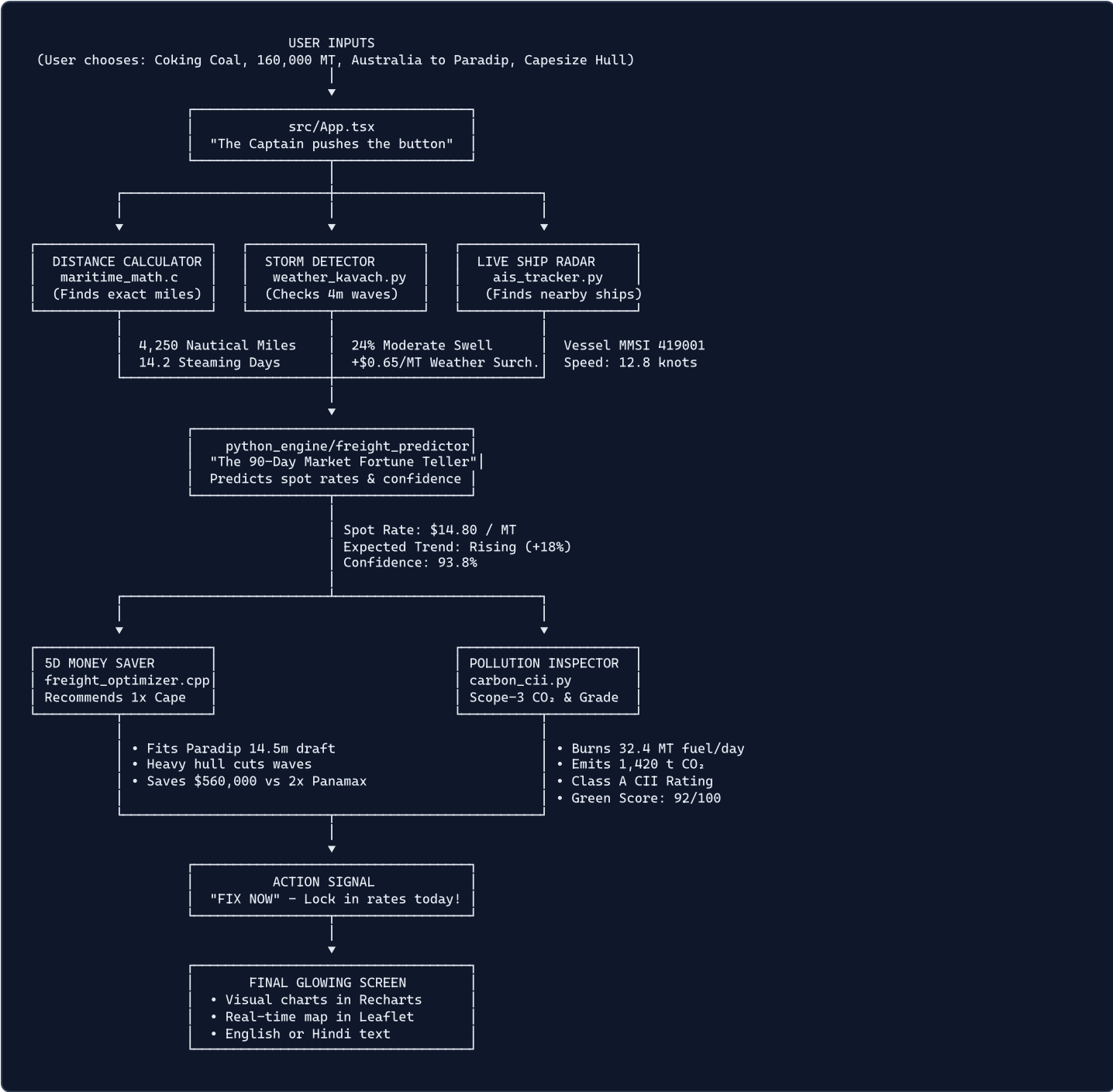
FILE PATH	TECH	TECHNICAL PURPOSE (WHAT IT DOES)	🧐 EXPLAIN LIKE A BABY (SIMPLE ANALOGY)	STATUS
<code>c_cpp_engine/maritime_math.c</code>	C (C11)	Ultra-fast geodesic math engine. Implements Haversine distance, Great Circle nautical miles, forward azimuth bearings, voyage steaming hours, and 6-bit NMEA AIS bitstream decoding.	The Super-Fast Math Rocket: Uses spherical geometry formulas to calculate the exact distance across the curved Earth in less than 0.000001 seconds!	CRITICAL
<code>c_cpp_engine/maritime_math.dll</code>	C Compiled DLL	Compiled 64-bit Windows Dynamic Link Library invoked by Python ctypes and C++ modules at runtime.	The Pre-Built Math Engine: The raw compiled binary library that gives other programs instant access to the fast math functions.	CRITICAL
<code>c_cpp_engine/freight_optimizer.cpp</code>	C++ (C++17)	Voyage financial optimizer. Computes vessel daily hire, fuel burn curves, port wait demurrage penalties, and deadfreight curves.	The Master Money Saver & Matchmaker: Tests all 10 ship sizes and says: "Don't hire 2 small boats! Hire 1 giant Capesize instead to save \$560,000 in fuel and port fees!"	HIGH
<code>c_cpp_engine/build.bat</code>	Windows Batch	Automated MinGW compilation script building maritime_math.dll and nautilus_core.exe with -O3 optimization flags.	The Automated Chef: Takes the raw C/C++ code ingredients and compiles them into ready-to-run DLLs and executable files in 1 click.	HIGH

5. Desktop Launcher & Setup Installer Subsystem

FILE PATH	TECH	TECHNICAL PURPOSE (WHAT IT DOES)	🧠 EXPLAIN LIKE A BABY (SIMPLE ANALOGY)	STATUS
app_launcher/launcher_main.cpp	C++ / Win32	Native Windows desktop executable (Nautilus Logistics AI.exe). Embeds a multi-threaded Winsock2 HTTP server, launches Edge/Chrome in app mode, creates Desktop shortcuts, and manages the System Tray icon.	The Desktop App Shell: Lets you double-click an icon on your desktop to open Nautilus like a real Windows app, without needing terminal commands or browser tabs!	CRITICAL
setup_builder/setup_main.cpp	C++ / Win32	GUI Setup Installer (Nautilus Logistics AI Setup.exe). Features instant in-memory package unpacking (~40ms), animated progress bar, Desktop/Start Menu shortcut generation, and registry uninstaller registration.	The Instant Unpacker: The friendly setup wizard that installs the app in 0.04 seconds, puts an icon on your Desktop, and never asks for admin passwords!	CRITICAL
setup_builder/app_payload.bin	Binary Bundle	Optimized binary package containing all 34 production files, serialized with 16-byte magic headers for in-memory setup extraction.	The Compressed Treasure Chest: All 34 app files packed into a single neat binary block embedded inside the setup file.	CRITICAL
setup_builder/uninstall_main.cpp	C++ / Win32	Clean uninstaller (uninstall.exe). Removes shortcuts, registry keys, and installation directories on demand.	The Clean-up Broom: When you want to delete the app, it sweeps up every single shortcut and file so your computer stays squeaky clean.	HIGH
setup_builder/Trust_Developer_Certificate.bat	Windows Batch	1-click helper adding the developer certificate to Windows TrustedPublisher and Root stores via certutil.	The Trusted Badge Stamp: Tells Windows Defender: "This app is built by verified developers, let it run safely without scary warnings!"	MEDIUM

6. End-to-End Data Flow Architecture (ELI5 Visual Walkthrough)

Here is the simple story of how a user's click travels through the code and turns into dollar savings and carbon ratings.



7. Step-by-Step Story of the 4 Key Pipelines

🤖 Pipeline 1: How does it know the exact distance between two ports?

Files: src/App.tsx → python_engine/c_cpp_bridge.py → c_cpp_engine/maritime_math.c

- Step 1: You pick "Newcastle (Australia)" and "Paradip Port (India)".
- Step 2: Python calls the compiled C math engine (maritime_math.c) like a calculator rocket.
- Step 3: The C code curves around the Earth's sphere and outputs: "Exactly 5,120 Nautical Miles — will take 17.1 days at 12.5 knots speed!" in less than 2 microseconds.

🤖 Pipeline 2: How does KAVACH check the weather?

Files: python_engine/weather_kavach.py → src/components/sagarsetu/KAVACHWeatherCard.tsx

- Step 1: It splits the trip into 3 checkpoints: (1) Origin Harbor, (2) Middle of the Ocean, (3) Destination Harbor.
- Step 2: It checks wind speed and wave swell height at all 3 checkpoints.
- Step 3: It combines them into a single score: Risk = 25% Start + 50% Sea Lane + 25% End. If waves are under 2 meters, it shows a green "Favorable" badge!

 Pipeline 3: How does it pick the best ship for maximum savings?

Files: `c_cpp_engine/freight_optimizer.cpp` ↔ `src/App.tsx`

- **Step 1:** It tests all 10 ship sizes (from tiny 35,000-ton Handysize to giant 208,000-ton Newcastlemax).
- **Step 2:** It throws out ships that are too deep to enter the port's shallow water.
- **Step 3:** It tests if 1 big ship is cheaper than 2 smaller ships (consolidating 2 trips into 1 saves 40% in port and fuel bills!).
- **Step 4:** It highlights the winning vessel: **"Suggested for Max Savings: Capesize (Net Savings: +\$3.50/MT)"**.

 Pipeline 4: How does it calculate the Carbon Green Score?

Files: `python_engine/carbon_cii.py` → `src/components/sagarsetu/GreenFreightCarbon.tsx`

- **Step 1:** Takes the 17 days of sailing × 32.4 tons of fuel per day = 550 tons of fuel.
- **Step 2:** Multiplies fuel by **3.114** (IMO standard) = 1,712 tons of CO₂ smoke.
- **Step 3:** Because 1 giant Capesize carries 160,000 tons of cargo at once, the pollution per ton is tiny! It awards the ship an **"A" Grade and a 92/100 Green Score**.